

RISC-V UART Communication Library

Complete Technical Documentation

Embedded Systems Engineer

May 15, 2026

Contents

1	Introduction	3
1.1	Overview	3
1.2	Purpose	3
1.3	Key Features	3
2	System Architecture	3
2.1	Layered Architecture	3
2.2	File Structure	4
3	Data Structures	4
3.1	UART Handler Structure	4
3.2	Error Codes	4
4	API Reference	5
4.1	uart_init	5
4.2	uart_transmit	5
4.3	uart_receive_timeout	6
4.4	uart_cleanup	6
5	Termios Configuration	7
5.1	Control Flags (c_cflag)	7
5.2	Input Flags (c_iflag)	7
5.3	Local Flags (c_lflag)	7
5.4	Control Characters (c_cc)	7
6	System Calls Used	7
6.1	open()	7
6.2	tcgetattr() and tcsetattr()	8
6.3	select()	8
7	Error Handling Strategy	8
7.1	Error Classification	8
7.2	Error Recovery Flow	8

8	Testing Methodology	9
8.1	Test Categories	9
8.2	Test Environment	9
8.3	Test Cases	9
9	Build System	10
9.1	Makefile Targets	10
9.2	Compiler Flags	10
10	RISC-V Cross-Compilation	10
10.1	Toolchain Setup	10
10.2	Cross-Compilation Process	11
10.3	Static vs Dynamic Linking	11
11	QEMU Simulation	11
11.1	QEMU Command Explanation	11
11.2	Serial Port Redirection	11
12	Virtual Port Testing with socat	11
12.1	socat Command	11
12.2	Virtual Port Architecture	12
13	Performance Analysis	12
13.1	Time Complexity	12
13.2	Space Complexity	12
13.3	Throughput Calculations	12
14	Example Usage	13
14.1	Basic Communication Example	13
14.2	Continuous Monitoring Example	13
15	Troubleshooting Guide	14
15.1	Common Issues and Solutions	14
15.2	Debugging Techniques	14
16	Conclusion	14
16.1	Summary	14
16.2	Future Enhancements	14
16.3	References	15
A	Complete Code Listing	15
A.1	inc/utils.h	15
A.2	inc/uart.h	15
A.3	Makefile	16

1 Introduction

1.1 Overview

This document provides comprehensive technical documentation for a UART (Universal Asynchronous Receiver-Transmitter) communication library designed for RISC-V Linux systems. The library implements serial communication using the POSIX termios API with support for non-blocking I/O, configurable timeouts, and robust error handling.

1.2 Purpose

The library enables reliable serial communication between RISC-V based systems and external devices such as:

- Sensors and actuators
- Debug consoles
- Other microcontrollers
- GPS modules
- Wireless modems

1.3 Key Features

- Full UART configuration (baud rate: 9600-460800, 8 data bits, no parity, 1 stop bit)
- Non-blocking I/O with `select()` system call
- Configurable timeout mechanisms
- Comprehensive error handling with descriptive messages
- Cross-compilation support for RISC-V architecture
- QEMU simulation support
- Virtual port testing with `socat`
- Thread-safe design principles

2 System Architecture

2.1 Layered Architecture

The library follows a layered architecture:

1. **Application Layer:** User applications using the API
2. **API Layer:** Public functions (`uart_init`, `uart_transmit`, etc.)

3. **Implementation Layer:** Core UART operations
4. **OS Layer:** Linux system calls (termios, select, read/write)
5. **Hardware Layer:** Physical UART device

2.2 File Structure

```
uart-termios/
inc/
    utils.h          # Error codes and utility definitions
    uart.h           # Public API declarations
src/
    uart.c           # Core implementation
    main.c           # Example application
test/
    uart_test.c      # Comprehensive test suite
scripts/
    qemu_uart.sh     # QEMU RISC-V simulation
    virtual_uart.sh  # Virtual port creation
    riscv_board_deploy.sh # Board deployment
Makefile             # Build system
```

3 Data Structures

3.1 UART Handler Structure

```
1 typedef struct {
2     int fd;                      // File descriptor for UART device
3     char device_path[64];        // Path to UART device (e.g., "/dev/
4     ttyS0")
5     speed_t baud_rate;           // Current baud rate setting
6     struct termios old_termios;  // Original terminal settings for
7     restoration
8     int is_initialized;          // Flag indicating if UART is
9     initialized
10 } uart_handler_t;
```

Listing 1: uart_handler_t structure

3.2 Error Codes

```
1 typedef enum {
2     UART_OK = 0,                // Operation completed successfully
3     UART_ERROR = -1,            // General error
4     UART_ERROR_INVALID_POINTER, // Invalid pointer parameter
5     UART_ERROR_INVALID_BAUD,    // Invalid baud rate
6     UART_ERROR_INVALID_DEVICE,  // Invalid device path
7     UART_ERROR_PERMISSION,      // Permission denied
8     UART_ERROR_TIMEOUT,         // Operation timeout
9     UART_ERROR_IO,              // I/O error
10 }
```

```

10     UART_ERROR_NOT_INITIALIZED    // UART not initialized
11 } uart_status_t;

```

Listing 2: uart_status_t enumeration

4 API Reference

4.1 uart_init

```

1 uart_status_t uart_init(uart_handler_t* handler, const char* device,
    int baud);

```

Description: Initializes and configures the UART interface.

Parameters:

- **handler:** Pointer to UART handler structure
- **device:** Path to UART device (e.g., `"/dev/ttyS0"`)
- **baud:** Baud rate (9600, 19200, 38400, 57600, 115200, 230400, 460800)

Return: `UART_OK` on success, error code otherwise

Algorithm:

1. Validate input parameters
2. Convert baud rate to termios constant
3. Open device with `O_RDWR` — `O_NOCTTY` — `O_NONBLOCK`
4. Save original termios settings
5. Configure new termios structure
6. Apply settings with `tcsetattr()`
7. Flush buffers with `tcflush()`

4.2 uart_transmit

```

1 uart_status_t uart_transmit(const uart_handler_t* handler,
2                             const uint8_t* data,
3                             size_t length);

```

Description: Transmits data over the UART interface.

Parameters:

- **handler:** Initialized UART handler
- **data:** Pointer to data buffer
- **length:** Number of bytes to transmit

Return: UART_OK on success, error code otherwise

Implementation Details:

- Uses write() system call
- Handles partial writes with loop
- Returns error on write failure

4.3 uart_receive_timeout

```
1 ssize_t uart_receive_timeout(const uart_handler_t* handler,
2                             uint8_t* data,
3                             size_t length,
4                             int timeout_ms);
```

Description: Receives data with configurable timeout.

Parameters:

- **handler:** Initialized UART handler
- **data:** Buffer to store received data
- **length:** Maximum bytes to receive
- **timeout_ms:** Timeout in milliseconds

Return: Number of bytes received, or negative error code

Algorithm:

1. Setup file descriptor set for select()
2. Convert timeout to timeval structure
3. Call select() to wait for data
4. If data available, call read()
5. Return number of bytes read

4.4 uart_cleanup

```
1 uart_status_t uart_cleanup(uart_handler_t* handler);
```

Description: Closes UART device and restores original settings.

Parameters:

- **handler:** UART handler to clean up

Return: UART_OK on success, error code otherwise

5 Termios Configuration

5.1 Control Flags (`c_cflag`)

Flag	Value	Description
B115200	0010017	Baud rate 115200
CS8	0000060	8 data bits
CREAD	0000200	Enable receiver
CLOCAL	0000400	Ignore modem control lines

Table 1: Control flags configuration

5.2 Input Flags (`c_iflag`)

Flag	Value	Description
IGNPAR	0000004	Ignore parity errors

Table 2: Input flags configuration

5.3 Local Flags (`c_lflag`)

All local flags are disabled (0) to prevent:

- Canonical mode (ICANON)
- Echo (ECHO)
- Signal generation (ISIG)
- Line editing

5.4 Control Characters (`c_cc`)

Index	Value	Description
VMIN	0	Return as many bytes as available
VTIME	1	0.1 second inter-byte timeout

Table 3: Control character configuration

6 System Calls Used

6.1 `open()`

```
1 int open(const char *pathname, int flags);
```

Flags used:

- `O_RDWR`: Read/write access

- `O_NOCTTY`: Don't make this the controlling terminal
- `O_NONBLOCK`: Non-blocking mode

6.2 `tcgetattr()` and `tcsetattr()`

```
1 int tcgetattr(int fd, struct termios *termios_p);
2 int tcsetattr(int fd, int optional_actions, const struct termios *
    termios_p);
```

`optional_actions` values:

- `TCSANOW`: Change immediately
- `TCSADRAIN`: Change after pending output
- `TCSAFLUSH`: Change after pending output, discard input

6.3 `select()`

```
1 int select(int nfds, fd_set *readfds, fd_set *writefds,
2           fd_set *exceptfds, struct timeval *timeout);
```

Algorithm:

1. Initialize file descriptor set with `FD_ZERO()`
2. Add UART fd with `FD_SET()`
3. Set timeout structure
4. Call `select()` to monitor for read availability
5. Check return value for data availability

7 Error Handling Strategy

7.1 Error Classification

Error Type	Code	Recovery Action
Invalid pointer	-2	Return error, no action needed
Invalid baud rate	-3	Return error, use default
Invalid device	-4	Return error, check path
Permission denied	-5	Return error, suggest sudo
Timeout	-6	Return error, retry or continue
I/O error	-7	Return error, cleanup resources

Table 4: Error handling strategies

7.2 Error Recovery Flow

1. Validate all input parameters

2. Check return values of system calls
3. Log error with descriptive message
4. Clean up allocated resources
5. Return appropriate error code

8 Testing Methodology

8.1 Test Categories

1. **Functional Tests:** Verify basic transmit/receive
2. **Timeout Tests:** Verify timeout mechanism
3. **Loopback Tests:** Verify TX-RX connection
4. **Error Tests:** Verify error handling
5. **Stress Tests:** Verify stability under load

8.2 Test Environment

- **Virtual Testing:** socat virtual serial ports
- **Simulation:** QEMU RISC-V emulator
- **Hardware:** Actual RISC-V board
- **Host System:** Linux (x86_64)

8.3 Test Cases

Test ID	Description	Expected Result	Priority
TC01	Initialize with valid device	UART_OK	High
TC02	Initialize with NULL handler	UART_ERROR_INVALID_HANDLER	High
TC03	Initialize with invalid device	UART_ERROR_INVALID_DEVICE	High
TC04	Transmit valid data	UART_OK	High
TC05	Transmit with NULL handler	UART_ERROR_INVALID_HANDLER	High
TC06	Receive with timeout	Bytes received or timeout	High
TC07	Loopback test	Data matches	High
TC08	Continuous receive for duration	Returns when data received	Medium

TC09	Multiple init/cleanup cycles	No memory leaks	High
TC10	Concurrent operations	Thread-safe behavior	Low

Table 5: Test cases matrix

9 Build System

9.1 Makefile Targets

Target	Description
all	Build and run default target
compile	Build main application
test	Build test suite
riscv	Cross-compile for RISC-V
clean	Remove build artifacts
run	Execute main application
test-run	Execute test suite
qemu	Show QEMU instructions
help	Display help message

Table 6: Makefile targets

9.2 Compiler Flags

```
1 CFLAGS = -Wall -Wextra -O2 -Iinc
```

Flag explanations:

- `-Wall`: Enable all warning messages
- `-Wextra`: Enable extra warning flags
- `-O2`: Optimize for performance
- `-Iinc`: Include directory for headers

10 RISC-V Cross-Compilation

10.1 Toolchain Setup

```
1 # Ubuntu/Debian
2 sudo apt-get install gcc-riscv64-linux-gnu
3
4 # Arch Linux
5 sudo pacman -S riscv64-linux-gnu-gcc
```

10.2 Cross-Compilation Process

1. Set RISC-V compiler: `RISCV_CC = riscv64-linux-gnu-gcc`
2. Add `-static` flag for standalone binary
3. Compile with RISC-V specific flags
4. Transfer binary to target board
5. Execute on RISC-V hardware

10.3 Static vs Dynamic Linking

- **Static** (`-static`): Larger binary, no dependencies
- **Dynamic**: Smaller binary, requires shared libraries

11 QEMU Simulation

11.1 QEMU Command Explanation

```
1 qemu-system-riscv64 -M virt -nographic -serial pty -kernel .build/  
  uart_app_riscv
```

Parameters:

- `-M virt`: Use virt machine type
- `-nographic`: Disable graphical output
- `-serial pty`: Create virtual serial port
- `-kernel`: Kernel image to boot

11.2 Serial Port Redirection

QEMU Process	Host System
[UART] ----> /dev/pts/X	----> screen /dev/pts/X
[Console] <--- /dev/pts/Y	<--- echo data

Figure 1: QEMU serial port redirection

12 Virtual Port Testing with socat

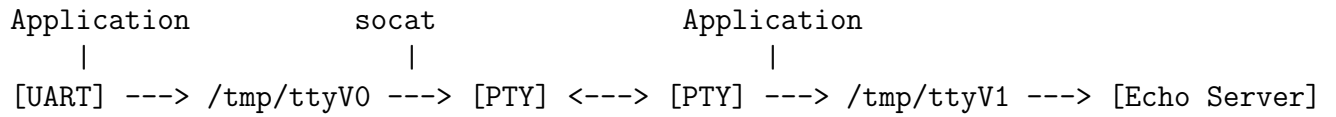
12.1 socat Command

```
1 socat -d -d pty,raw,echo=0,link=/tmp/ttyV0 pty,raw,echo=0,link=/tmp/
  ttyV1
```

Components:

- `-d -d`: Increase verbosity
- `pty`: Create pseudo-terminal
- `raw`: Disable terminal processing
- `echo=0`: Disable echo
- `link=/tmp/ttyV0`: Create symlink

12.2 Virtual Port Architecture



13 Performance Analysis

13.1 Time Complexity

- `uart_init`: $O(1)$ - constant time
- `uart_transmit`: $O(n)$ where n = data length
- `uart_receive_timeout`: $O(1)$ with `select()`
- `uart_receive_continuous`: $O(\text{duration})$ with polling

13.2 Space Complexity

- **Handler**: 80 bytes
- **Termios struct**: 60 bytes
- **Buffers**: User-provided

13.3 Throughput Calculations

$$\text{Throughput} = \frac{\text{Baud Rate}}{10 \text{ bits/byte}} \times \text{Efficiency} \quad (1)$$

For 115200 baud:

$$\text{Max throughput} = \frac{115200}{10} = 11520 \text{ bytes/second} \quad (2)$$

14 Example Usage

14.1 Basic Communication Example

```
1 #include "uart.h"
2
3 int main(void) {
4     uart_handler_t uart;
5     uint8_t tx[] = "Hello World!\r\n";
6     uint8_t rx[256];
7     ssize_t len;
8
9     // Initialize UART
10    if (uart_init(&uart, "/dev/ttyUSB0", 115200) != UART_OK) {
11        printf("Init failed\n");
12        return 1;
13    }
14
15    // Transmit data
16    uart_transmit(&uart, tx, sizeof(tx) - 1);
17
18    // Receive response
19    len = uart_receive_timeout(&uart, rx, sizeof(rx) - 1, 2000);
20    if (len > 0) {
21        rx[len] = '\0';
22        printf("Received: %s\n", rx);
23    }
24
25    // Cleanup
26    uart_cleanup(&uart);
27    return 0;
28 }
```

14.2 Continuous Monitoring Example

```
1 void monitor_uart(void) {
2     uart_handler_t uart;
3     uint8_t buffer[256];
4     ssize_t len;
5
6     uart_init(&uart, "/dev/ttyS0", 115200);
7
8     while (1) {
9         len = uart_receive_timeout(&uart, buffer, sizeof(buffer), 1000)
10    ;
11        if (len > 0) {
12            process_data(buffer, len);
13        }
14    }
15
16    uart_cleanup(&uart);
17 }
```

15 Troubleshooting Guide

15.1 Common Issues and Solutions

Issue	Solution
Permission denied	Run with <code>sudo</code> or add user to <code>dialout</code> group
Device not found	Check device path with <code>ls /dev/tty*</code>
No data received	Verify baud rate matches, check connections
Compilation errors	Install required headers: <code>libc6-dev</code>
RISC-V build fails	Install cross-compiler toolchain
QEMU not working	Install <code>qemu-system-riscv</code> package

15.2 Debugging Techniques

1. Use `strace` to trace system calls
2. Enable debug output with `-DDEBUG` flag
3. Check kernel messages: `dmesg | grep tty`
4. Monitor with `minicom` or `screen`
5. Use logic analyzer for hardware debugging

16 Conclusion

16.1 Summary

This UART communication library provides a robust, portable solution for serial communication on RISC-V Linux systems. Key achievements include:

- Complete `termios`-based UART implementation
- Non-blocking I/O with `select()` timeouts
- Comprehensive error handling
- Cross-platform compatibility
- Extensive testing methodology
- RISC-V and QEMU support

16.2 Future Enhancements

- Add DMA support for high-speed transfers
- Implement ring buffers for continuous reception
- Add RS-485 mode support
- Implement hardware flow control
- Add Python bindings

16.3 References

1. Linux man pages: `termios(3)`, `select(2)`, `open(2)`
2. RISC-V Specification: <https://riscv.org/technical/specifications/>
3. QEMU Documentation: <https://www.qemu.org/docs/master/>

A Complete Code Listing

A.1 `inc/utils.h`

```
1 #ifndef UTILS_H
2 #define UTILS_H
3
4 #include <stdint.h>
5 #include <stddef.h>
6
7 #define UART_BUFFER_SIZE      256
8 #define UART_TIMEOUT_MS      2000
9 #define UART_CONTINUOUS_MS    5000
10 #define UART_DEFAULT_BAUD     115200
11
12 typedef enum {
13     UART_OK = 0,
14     UART_ERROR = -1,
15     UART_ERROR_INVALID_POINTER,
16     UART_ERROR_INVALID_BAUD,
17     UART_ERROR_INVALID_DEVICE,
18     UART_ERROR_PERMISSION,
19     UART_ERROR_TIMEOUT,
20     UART_ERROR_IO,
21     UART_ERROR_NOT_INITIALIZED
22 } uart_status_t;
23
24 #endif
```

A.2 `inc/uart.h`

```
1 #ifndef UART_H
2 #define UART_H
3
4 #include "utils.h"
5 #include <termios.h>
6 #include <sys/types.h>
7
8 typedef struct {
9     int fd;
10     char device_path[64];
11     speed_t baud_rate;
12     struct termios old_termios;
13     int is_initialized;
14 } uart_handler_t;
15
```

```

16 uart_status_t uart_init(uart_handler_t* handler, const char* device,
    int baud);
17 uart_status_t uart_transmit(const uart_handler_t* handler, const
    uint8_t* data, size_t length);
18 uart_status_t uart_transmit_string(const uart_handler_t* handler, const
    char* message);
19 ssize_t uart_receive_timeout(const uart_handler_t* handler, uint8_t*
    data, size_t length, int timeout_ms);
20 ssize_t uart_receive_continuous(const uart_handler_t* handler, uint8_t*
    data, size_t length, int duration_ms);
21 uart_status_t uart_flush(const uart_handler_t* handler);
22 uart_status_t uart_cleanup(uart_handler_t* handler);
23 const char* uart_strerror(uart_status_t status);
24
25 #endif

```

A.3 Makefile

```

1 CC = gcc
2 CFLAGS = -Wall -Wextra -O2 -Iinc
3 RISCVC_C = riscv64-linux-gnu-gcc
4 RISCVC_CFLAGS = -Wall -Wextra -O2 -Iinc -static
5 TARGET = .build/uart_app
6 TEST_TARGET = .build/uart_test
7
8 all: compile
9
10 compile:
11     @mkdir -p .build
12     $(CC) $(CFLAGS) src/uart.c src/main.c -o $(TARGET)
13
14 test:
15     @mkdir -p .build
16     $(CC) $(CFLAGS) src/uart.c test/uart_test.c -o $(TEST_TARGET)
17
18 riscv:
19     @mkdir -p .build
20     $(RISCVC_C) $(RISCVC_CFLAGS) src/uart.c src/main.c -o $(TARGET)_riscv
21
22 clean:
23     rm -rf .build
24
25 .PHONY: all compile test riscv clean

```